

Line-by-Line Code Analysis: analyzer.py

Detailed walk-through of Suspect Evaluation and RAG-based Risk Scoring Engine

PART 1: IMPORTS, CLASS INITIALIZATION & CASE RELOADING

Line	Source Code Snippet	Detailed Explanation
1-2	<pre>from typing import Dict, Any, List from rag.retriever import CaseRetriever</pre>	Imports type hints for dictionary and list definitions, along with the custom <code>CaseRetriever</code> class from the internal RAG module.
5	<pre>class DetectiveAgent:</pre>	Defines the core <code>DetectiveAgent</code> class responsible for evaluating suspect behavior and analyzing historical case similarity.
7-9	<pre>def __init__(self): # Create the RAG retriever when... self.retriever = CaseRetriever()</pre>	The constructor method initializes the instance variable <code>self.retriever</code> with an instance of <code>CaseRetriever</code> for vector searches.
11-13	<pre>def reload_cases(self) -> int: """Refresh indexed cases...""" return self.retriever.reload()</pre>	Exposes a method to refresh vector database indexes when new JSON case files are uploaded, returning the count of reloaded records.

PART 2: QUERY CONSTRUCTION & VECTOR SIMILARITY RETRIEVAL

Line	Source Code Snippet	Detailed Explanation
15-21	<pre>def evaluate_suspect(self, name: str, behavior: str, mo_suspected: str, personality_notes: str) -> Dict[str, Any]:</pre>	Primary method that accepts suspect details and returns a structured dictionary containing risk score, risk level, and matching evidence.
24-29	<pre>query_parts = [] for value in (behavior, mo_suspected, per sonality_notes): value_text = str(value or "").strip() if value_text and value_text not in query _parts: query_parts.append(value_text)</pre>	Iterates over behavioral input fields, strips empty strings, and deduplicates inputs to form a clean search query tuple.
30	<pre>query_str = "; ".join(query_parts)</pre>	Joins non-empty query components into a single semicolon-delimited text string for vector retrieval.
33-36	<pre>retrieved_cases = self.retriever.search_s imilar_cases(query_str, top_k=3)</pre>	Executes vector search against the RAG case index using <code>query_str</code> , requesting the top 3 most relevant historical cases.
40-47	<pre>MATCH_DISTANCE_THRESHOLD = 0.80 matched_cases: List[Dict[str, Any]] = [] if retrieved_cases: for case in retrieved_cases: dist = case.get("distance") if dist is not None and float(dist) <= MA TCH_DISTANCE_THRESHOLD: matched_cases.append(case)</pre>	Establishes a maximum vector distance threshold of 0.80 (where smaller distance = higher similarity) and filters out weak matches.

PART 3: DYNAMIC SCORING LOGIC & KEYWORD FALLBACKS

Line	Source Code Snippet	Detailed Explanation
50	<pre>base_score = 15</pre>	Sets baseline risk score to 15% before factoring in case vector similarity or severe behavioral keywords.
52-64	<pre>if matched_cases: similarities = [float(case.get("similarit y", 0.0)) for case in matched_cases] strongest_similarity = max(similarities, default=0.0) average_similarity = sum(similarities) / len(similarities)...</pre>	Extracts individual cosine similarity metrics from matched cases and calculates both peak (strongest) and mean (average) values.
66-73	<pre>similarity_score = ((strongest_similarity * 60) + (average_similarity * 25)) score = round(min(95, max(15, base_score + similarity_score)))</pre>	Computes weighted similarity score (60% peak + 25% average) and clamps final score strictly between 15% and 95%.
75-81	<pre>if strongest_similarity >= 0.55: match_qu ality = "Strong..." elif strongest_similarity >= 0.35: match_ quality = "Moderate..." else: match_quality = "Weak..."</pre>	Assigns a human-readable match quality tier based on peak similarity thresholds.
83-99	<pre>else: combined_text = f"{behavior} {mo_suspecte d} {personality_notes}".lower() severe_keywords = ["murder", "kill", "wea pon", ...] kw_matches = sum(1 for kw in severe_keywo rds if kw in combined_text)</pre>	Fallback heuristic executed when no vector matches pass threshold: checks combined text against a list of severe crime keywords.
101-111	<pre>if kw_matches > 0: score = min(95, max(25, base_score + (kw_ matches * 15))) else: score = 15 match_quality = "No reliable case-index m atch"</pre>	Increases score by +15 per keyword match (clamped between 25% and 95%), or resets to 15% if no keywords are detected.

PART 4: RISK CATEGORIZATION & RESPONSE PACKAGING

Line	Source Code Snippet	Detailed Explanation
113-119	<pre>if score >= 70: risk_level = "HIGH RISK" elif score >= 40: risk_level = "MEDIUM RI SK" else: risk_level = "LOW RISK"</pre>	Categorizes numerical score into clear risk brackets: HIGH RISK (≥70%), MEDIUM RISK (40-69%), or LOW RISK (<40%).
121-133	<pre>return { "suspect_name": name, "tendency_score": f"{score}%", "risk_level": risk_level, "match_quality": match_quality, "summary": (...), "similar_cases": matched_cases }</pre>	Constructs and returns the final structured dictionary containing all computed risk metrics, summary text, and retrieved case objects.